

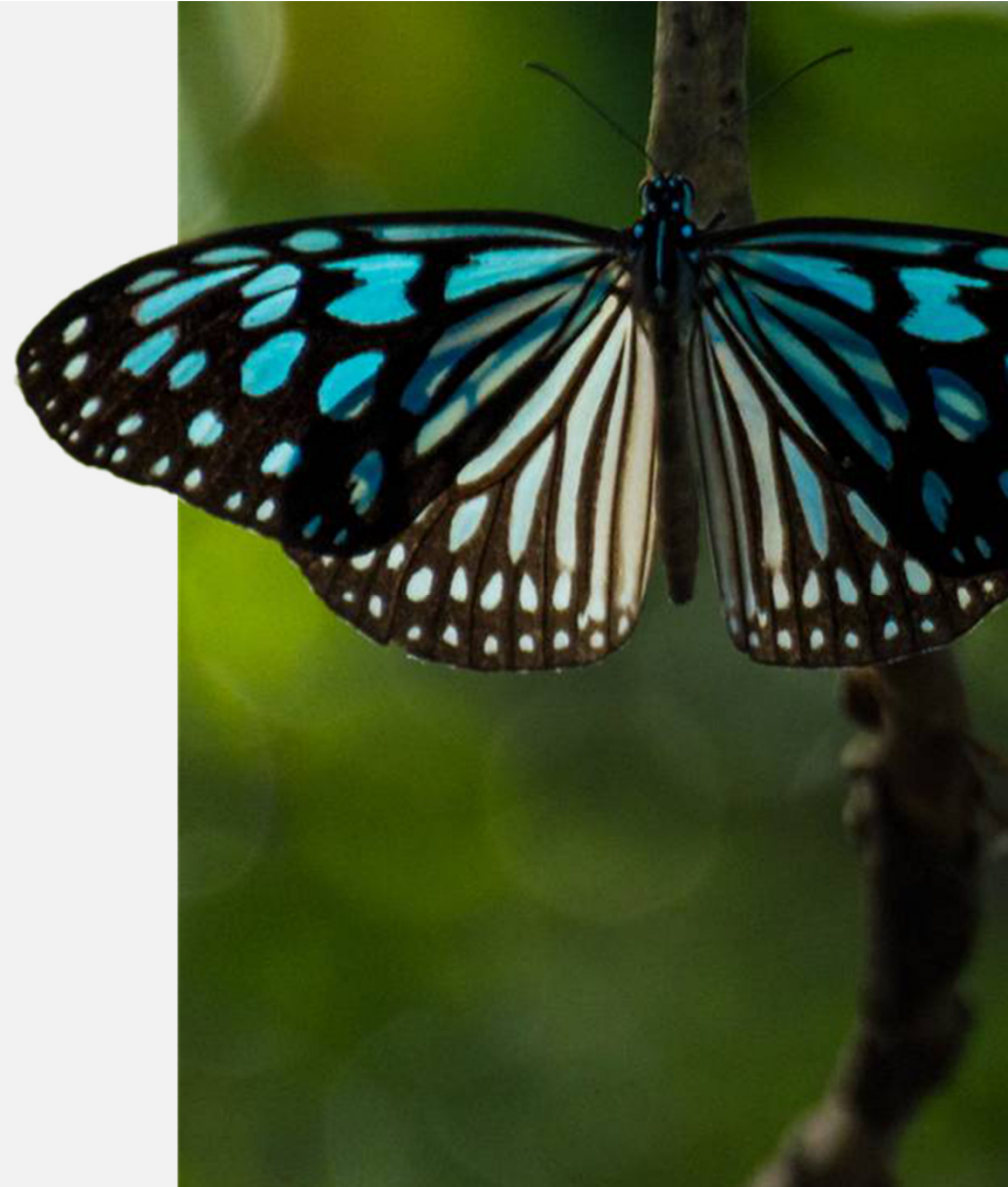


# **AWS APPLICATION INTEGRATION & ARCHITECTURAL BEST PRACTICES**

- Module 7
- BCSE355L – AWS Solution Architect

# Agenda

- AWS Application Integration
- Amazon SQS (Simple Queue Service)
- Amazon SNS (Simple Notification Service)
- Amazon SWF (Simple Workflow Service)
- AWS Step Functions
- AWS Well-Architected Framework
- Design Principles for AWS Cloud Architectures



# AWS APPLICATION INTEGRATION

- Application integration on AWS is a suite of services that enable communication between decoupled components within microservices, distributed systems, and serverless applications.
- You don't need to refactor your entire architecture to benefit—decoupling applications at any scale can reduce the impact of changes, making it easier to update and faster to release new features.

# AWS APPLICATION INTEGRATION

- **Develop with agility**
  - By using application integration services to connect your apps, you no longer need to write custom code to enable interoperability.
  - This also limits extra code that may be repeated in your microservices and functions.
- **Focus on innovation**
  - Application integration services shift your operational responsibilities to AWS, so you can focus on development and innovation.
  - With automatic scaling, you no longer need to provision, patch, and manage servers.

# AWS APPLICATION INTEGRATION

- **Scale and fail independently**
  - Decoupling your app with application integration services allows them to remain interoperable, but if one service has a failure or spike in workload, it won't affect the rest of them.
- **Message with confidence**
  - Application integration messaging services can accommodate any level of throughput and use cross-availability zone message storage to provide high availability and durability.

# AWS APPLICATION INTEGRATION

| Category              | What is it                                                                                                     | AWS service                                                     |
|-----------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <u>API Management</u> | Create, publish, maintain, monitor, and secure APIs at any scale for serverless workloads and web applications | <a href="#"><u>Amazon API Gateway</u></a>                       |
|                       | Create a flexible API to securely access, manipulate, and combine data from one or more data sources           | <a href="#"><u>AWS App Sync</u></a>                             |
| <u>Event Bus</u>      | Build an event-driven architecture that connects application data from your own apps, SaaS, and AWS services   | <a href="#"><u>Amazon EventBridge</u></a>                       |
| <u>Messaging</u>      | Reliable high throughput pub/sub, SMS, email, and mobile push notifications                                    | <a href="#"><u>Amazon Simple Notification Service (SNS)</u></a> |
|                       | Message queue that sends, stores, and receives messages between application components at any volume           | <a href="#"><u>Amazon Simple Queue Service (SQS)</u></a>        |
|                       | Message broker for Apache ActiveMQ and RabbitMQ that makes migration easy and enables hybrid architectures     | <a href="#"><u>Amazon MQ</u></a>                                |

# AWS APPLICATION INTEGRATION

| Category                | What is it                                                                                              | AWS service                                                               |
|-------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| No-code API Integration | Automate the flow of data between SaaS applications and AWS services at nearly any scale, without code. | <a href="#"><u>Amazon AppFlow</u></a>                                     |
| Workflows               | Coordinate multiple AWS services into serverless workflows so you can build and update apps quickly     | <a href="#"><u>AWS Step Functions</u></a>                                 |
|                         | Run Apache Airflow at scale without provisioning or managing infrastructure                             | <a href="#"><u>Amazon Managed Workflows for Apache Airflow (MWAA)</u></a> |
|                         | Automate transformation of EDI documents to JSON and XML to simplify downstream integrations            | <a href="#"><u>AWS B2B Data Interchange</u></a>                           |
| Automated Test Library  | Create and run automated integration tests to validate event-driven applications in the AWS Cloud       | <a href="#"><u>AWS Integrated Application Test Kit</u></a>                |

# AWS APPLICATION INTEGRATION





# AWS APPLICATION INTEGRATION

- **Services**
  - AWS Step Functions
  - Amazon AppFlow
  - AWS B2B Data Interchange
  - Amazon EventBridge
  - Amazon Managed Workflows for Apache Airflow (MWAA)
  - Amazon MQ
  - Amazon Simple Notification Service
  - Amazon Simple Queue Service
  - Amazon Simple Workflow Service



# **AMAZON STEP FUNCTION**

# AWS STEP FUNCTIONS

- AWS Step Functions is a fully managed service that makes it easy to coordinate the components of distributed applications and microservices using visual workflows.
- Building applications from individual components that each perform a discrete function lets you scale easily and change applications quickly.
- Step Functions is a reliable way to coordinate components and step through the functions of your application.
- Step Functions provides a graphical console to arrange and visualize the components of your application as a series of steps.

# AWS STEP FUNCTIONS

- This makes it simple to build and run multi-step applications.
- Step Functions automatically initiates and tracks each step, and retries when there are errors, so your application runs in order and as expected.
- Step Functions logs the state of each step, so when things do go wrong, you can diagnose and debug problems quickly.
- You can change and add steps without even writing code, so you can easily evolve your application and innovate faster.

# AWS STEP FUNCTIONS

- **Benefits of Step Functions**
  - **Quick Onboarding** - Start building quickly with Workflow Studio, a simple drag-and-drop interface for expressing complex business logic.
  - **Simple Automation** - Automate workflows across over 220 AWS services without maintaining code.
  - **Process data on demand** - Use code to process data on demand with large-scale parallel workflows
  - **Event-driven architecture visualization** - Visualize and develop resilient workflows for event-driven architectures.

# AWS STEP FUNCTIONS

- Step Functions is a visual workflow service that helps developers use AWS services to build distributed applications, automate processes, orchestrate microservices, and create data and machine learning (ML) pipelines.



# AWS STEP FUNCTIONS

- **Use cases**
  - **Automate extract, transform, and load (ETL) processes** - Ensure that multiple long-running ETL jobs run in order and complete successfully, without the need for manual orchestration.
  - **Orchestrate large-scale parallel workloads** - Iterate over and process large data-sets such as security logs, transaction data, or image and video files.
  - **Orchestrate microservices** - Combine multiple AWS Lambda functions into responsive serverless applications and microservices.
  - **Automate security and IT functions** - Create automated workflows, including manual approval steps, for security incident response.

# AMAZON APPFLOW

- Amazon AppFlow is a fully managed integration service that enables you to securely transfer data between Software-as-a-Service (SaaS) applications such as Salesforce, Zendesk, Slack, and ServiceNow, and AWS services such as Amazon S3 and Amazon Redshift, in just a few clicks.
- With Amazon AppFlow, you can run data flows at enterprise scale at the frequency you choose - on a schedule, in response to a business event, or on demand.
- You can configure data transformation capabilities such as filtering and validation to generate rich, ready-to-use data as part of the flow itself, without additional steps.
- Amazon AppFlow; automatically encrypts data in motion, and allows users to restrict data from flowing over the public internet for SaaS applications that are integrated with AWS PrivateLink, reducing exposure to security threats.



# AWS B2B DATA INTERCHANGE

- AWS B2B Data Interchange (B2Bi) automates the transformation of Electronic Data Interchange (EDI) documents into JSON and XML formats to simplify your downstream data integrations.
- Businesses use EDI documents to exchange transactional data with trading partners, such as suppliers and end customers, using standardized formats such as X12.
- With B2Bi, you can onboard and manage your trading partners and automate the transformation of EDI documents into common data representations such as JSON and XML using a low-code interface.
- This approach reduces the time, complexity, and cost associated with preparing and integrating EDI data into their business applications and purpose-built data lakes.

# AMAZON EVENTBRIDGE

- Amazon EventBridge is a serverless event bus that makes it easier to build event-driven applications at scale using events generated from your applications, integrated Software-as-a-Service (SaaS) applications, and AWS services.
- EventBridge delivers a stream of real-time data from event sources such as Zendesk or Shopify to targets such as AWS Lambda and other SaaS applications.
- You can set up routing rules to determine where to send your data to build application architectures that react in real-time to your data sources with event publisher and consumer completely decoupled.

# AMAZON MANAGED WORKFLOWS FOR APACHE AIRFLOW (MWAA)

- Amazon Managed Workflows for Apache Airflow (MWAA) is a managed orchestration service for Apache Airflow that makes it easier to set up and operate end-to-end data pipelines in the cloud at scale.
- Apache Airflow is an open-source tool used to programmatically author, schedule, and monitor sequences of processes and tasks referred to as "workflows."
- With Managed Workflows, you can use Airflow and Python to create workflows without having to manage the underlying infrastructure for scalability, availability, and security.
- Managed Workflows automatically scales its workflow capacity to meet your needs, and is integrated with AWS security services to help provide you with fast and secure access to data.

# AMAZON MQ

- Amazon MQ is a managed message broker service for Apache ActiveMQ Classic and RabbitMQ that makes it easy to set up and operate message brokers in the cloud.
- Message brokers allow different software systems—often using different programming languages, and on different platforms—to communicate and exchange information.
- Amazon MQ reduces your operational load by managing the provisioning, setup, and maintenance of ActiveMQ and RabbitMQ, popular open-source message brokers.
- Connecting your current applications to Amazon MQ is easy because it uses industry-standard APIs and protocols for messaging, including JMS, NMS, AMQP, STOMP, MQTT, and WebSocket.



# **AMAZON SIMPLE QUEUE SERVICE (SQS)**

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- Amazon SQS is a fast, reliable, scalable, and fully managed message queuing service.
- Amazon SQS makes it simple and cost effective to decouple the components of a cloud application.
- Amazon SQS, you can offload the administrative burden of operating and scaling a highly available messaging cluster while paying a low price for only what you use.
- *Amazon SQS queue* is basically a buffer between the application components that receive data and those components that process the data in your system.
- If your processing servers cannot process the work fast enough (perhaps due to a spike in traffic), the work is queued so that the processing servers can get to it when they are ready. This means that work is not lost due to insufficient resources.

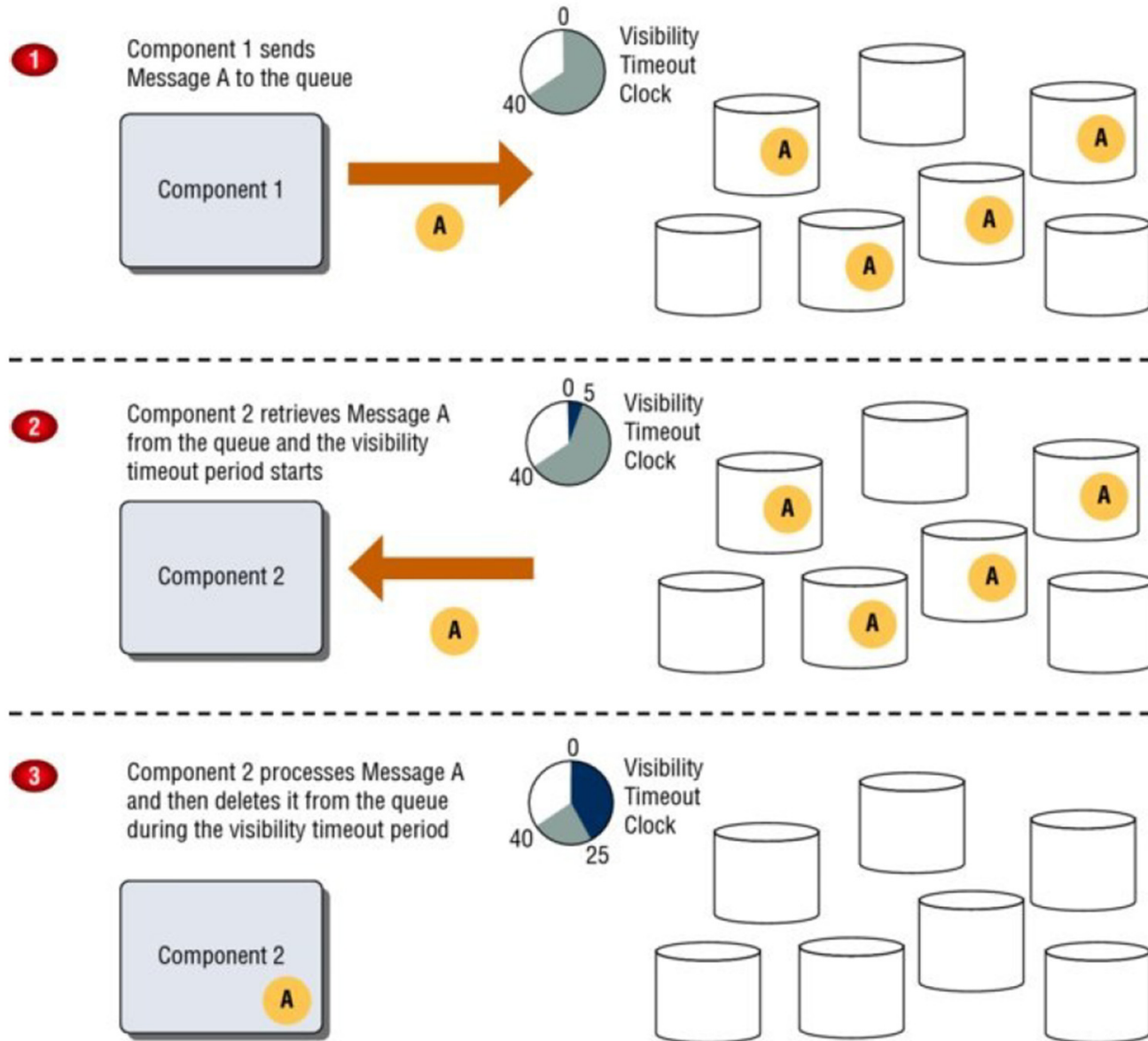
# AMAZON SIMPLE QUEUE SERVICE (SQS)

- Amazon SQS ensures delivery of each message at least once and supports multiple readers and writers interacting with the same queue.
- A single queue can be used simultaneously by many distributed application components, with no need for those components to coordinate with one another to share the queue.
- Although most of the time each message will be delivered to your application exactly once, you should design your system to be idempotent.
- Amazon SQS is engineered to be highly available and to deliver messages reliably and efficiently; however, the service does not guarantee First In, First Out (FIFO) delivery of messages.



## Message Lifecycle

- Lifecycle of an Amazon SQS message, called Message A, from creation to deletion. Assume that a queue already exists.





# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Message Lifecycle**
- Lifecycle of an Amazon SQS message, called Message A, from creation to deletion. Assume that a queue already exists.
  1. Component 1 sends Message A to a queue, and the message is redundantly distributed across the Amazon SQS servers.
  2. When Component 2 is ready to process a message, it retrieves messages from the queue, and Message A is returned. While Message A is being processed, it remains in the queue and is not returned to subsequently receive requests for the duration of the visibility timeout.
  3. Component 2 deletes Message A from the queue to prevent the message from being received and processed again after the visibility timeout expires.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Delay Queues and Visibility Timeouts**
  - Delay queues allow you to postpone the delivery of new messages in a queue for a specific number of seconds.
  - If you create a delay queue, any message that you send to that queue will be invisible to consumers for the duration of the delay period.
  - To create a delay queue, use `CreateQueue` and set the `DelaySeconds` attribute to any value between 0 and 900 (15 minutes).
  - You can also turn an existing queue into a delay queue by using `SetQueueAttributes` to set the queue's `DelaySeconds` attribute. The default value for `DelaySeconds` is 0.

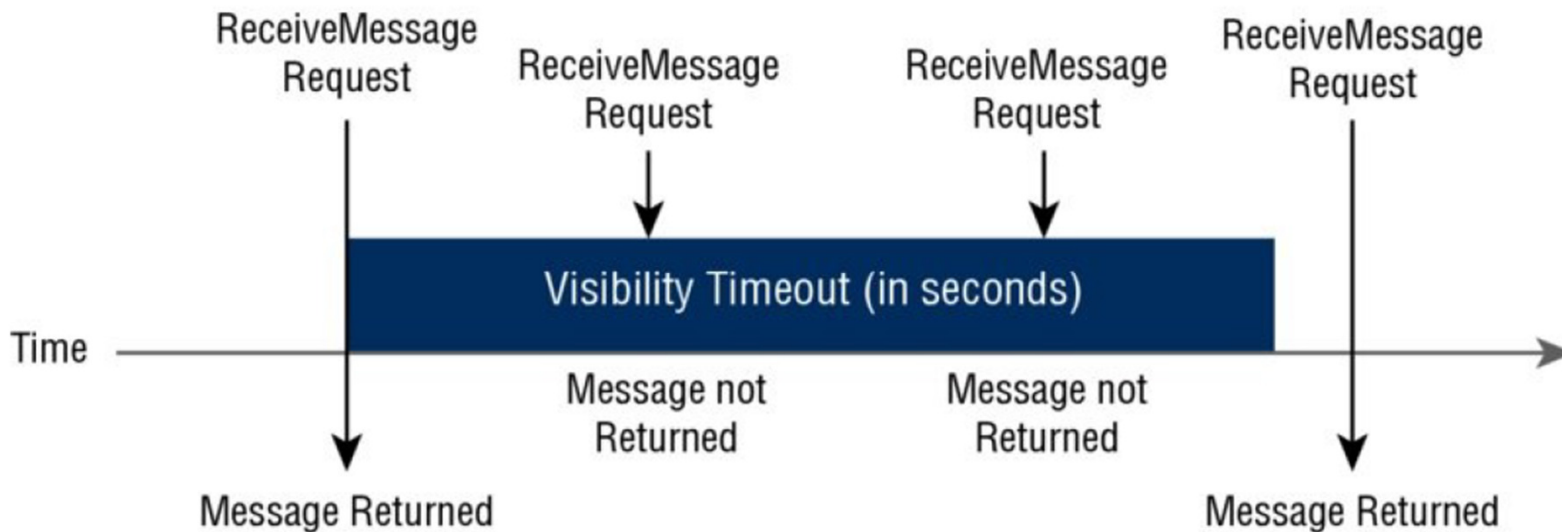
# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Delay Queues and Visibility Timeouts**

- Delay queues are similar to visibility timeouts in that both features make messages unavailable to consumers for a specific period of time.
- The difference is that a delay queue hides a message when it is first added to the queue, whereas a visibility timeout hides a message only after that message is retrieved from the queue.
- When a message is in the queue but is neither delayed nor in a visibility timeout, it is considered to be "in flight."
- You can have up to 120,000 messages in flight at any given time.
- Amazon SQS supports up to 12 hours' maximum visibility timeout.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- Visibility Timeouts



# AMAZON SIMPLE QUEUE SERVICE (SQS)

- Queue Operations, Unique IDs, and Metadata
  - The defined operations for Amazon SQS queues are CreateQueue, ListQueues, DeleteQueue, SendMessage, SendMessageBatch, ReceiveMessage, DeleteMessage, DeleteMessageBatch, PurgeQueue, ChangeMessageVisibility, ChangeMessageVisibilityBatch, SetQueueAttributes, GetQueueAttributes, GetQueueUrl, ListDeadLetterSourceQueues, AddPermission, and RemovePermission.
  - Only the AWS account owner or an AWS identity that has been granted the proper permissions can perform operations.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- Queue Operations, Unique IDs, and Metadata
  - Your messages are identified via a globally unique ID that Amazon SQS returns when the message is delivered to the queue.
  - The ID isn't required in order to perform any further actions on the message, but it's useful for tracking whether a particular message in the queue has been received.
  - When you receive a message from the queue, the response includes a receipt handle, which you must provide when deleting the message.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Queue and Message Identifiers**
  - Amazon SQS uses three identifiers that you need to be familiar with: **queue URLs, message IDs, and receipt handles.**
  - When creating a new queue, you must provide a queue name that is unique within the scope of all of your queues.
  - Amazon SQS assigns each queue an identifier **called a queue URL, which includes the queue name** and other components that Amazon SQS determines.
  - Whenever you want to perform an action on a queue, you must provide its queue URL.
  - Amazon SQS assigns each message **a unique ID that** it returns to you in the SendMessage response - useful for identifying messages, but note **that to delete a message, you need the message's receipt handle** instead of the message ID.
  - The maximum length of a message ID is 100 characters.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Message Attributes**

- Amazon SQS provides support for message attributes. Message attributes allow you to provide structured metadata items (such as timestamps, geospatial data, signatures, and identifiers) about the message.
- Message attributes are optional and separate from, but sent along with, the message body. The receiver of the message can use this information to help decide how to handle the message without having to process the message body first.



# AMAZON SIMPLE QUEUE SERVICE (SQS)

- Long Polling

- Application queries the Amazon SQS queue for messages, it calls the function `ReceiveMessage` - will check for the existence of a message in the queue and return immediately, either with or without a message.
- If your code makes periodic calls to the queue, this pattern is sufficient.
- If your SQS client is just a loop that repeatedly checks for new messages, however, then this pattern becomes problematic, as the constant calls to `ReceiveMessage` burn CPU cycles and tie up a thread.
- In this situation, use long polling that send a `WaitTimeSeconds` argument to `ReceiveMessage` of up to 20 seconds.
- If there is no message in the queue, then the call will wait up to `WaitTimeSeconds`. If a message appears before the time expires, the call will return the message right away.
- Long polling drastically reduces the amount of load on your client.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Dead Letter Queues**

- A dead letter queue is a queue that other (source) queues can target to send messages that for some reason could not be successfully processed.
- A primary benefit of using a dead letter queue is the ability to sideline and isolate the unsuccessfully processed messages.
- You can then analyze any messages sent to the dead letter queue to try to determine the cause of failure.

# AMAZON SIMPLE QUEUE SERVICE (SQS)

- **Access Control**
  - While IAM can be used to control the interactions of different AWS identities with queues, there are often times when you will want to expose queues to other accounts.
  - These situations may include:
    - You want to grant another AWS account a particular type of access to your queue.
    - You want to grant another AWS account access to your queue for a specific period of time.
    - You want to grant another AWS account access to your queue only if the requests come from your Amazon EC2 instances.
    - You want to deny another AWS account access to your queue.
  - While close coordination between accounts may allow these types of actions through the use of IAM roles, that level of coordination is frequently unfeasible



# **AMAZON SIMPLE NOTIFICATION SERVICE (SNS)**

# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- Amazon SNS is a web service for mobile and enterprise messaging that enables you to set up, operate, and send notifications.
- It is designed to make web-scale computing easier for developers.
- Amazon SNS follows the publish-subscribe (pub-sub) messaging paradigm, with notifications being delivered to clients using a push mechanism that eliminates the need to check periodically (or poll) for new information and updates.

# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

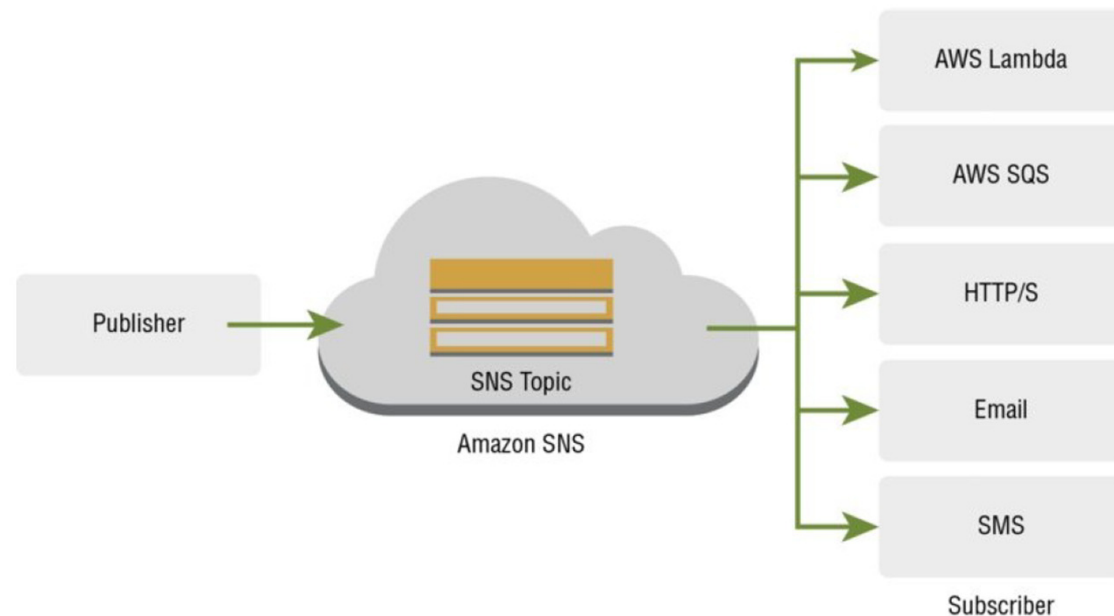
- Amazon SNS consists of two types of clients: publishers and subscribers (sometimes known as producers and consumers).
- Publishers communicate to subscribers asynchronously by sending a message to a topic.
- A topic is simply a logical access point/communication channel that contains a list of subscribers and the methods used to communicate to them.
- When you send a message to a topic, it is automatically forwarded to each subscriber of that topic using the communication method configured for that subscriber.

# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- When using Amazon SNS, you (as the owner) create a topic and control access to it by defining policies that determine which publishers and subscribers can communicate with the topic and via which technologies.
- Publishers send messages to topics that they created or that they have permission to publish to. Instead of including a specific destination address in each message, a publisher sends a message to the topic, and Amazon SNS delivers the message to each subscriber for that topic.
- Each topic has a unique name that identifies the Amazon SNS endpoint where publishers post messages and subscribers register for notifications.

# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- A publisher issues a message on a topic. The message is then delivered to the subscribers of that topic using different methods, such as Amazon SQS, HTTP, HTTPS, email, SMS, and AWS Lambda.





# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- When using Amazon SNS, you (as the owner) create a topic and control access to it by defining policies that determine which publishers and subscribers can communicate with the topic and via which technologies.
- Publishers send messages to topics that they created or that they have permission to publish to. Instead of including a specific destination address in each message, a publisher sends a message to the topic, and Amazon SNS delivers the message to each subscriber for that topic.
- Each topic has a unique name that identifies the Amazon SNS endpoint where publishers post messages and subscribers register for notifications.

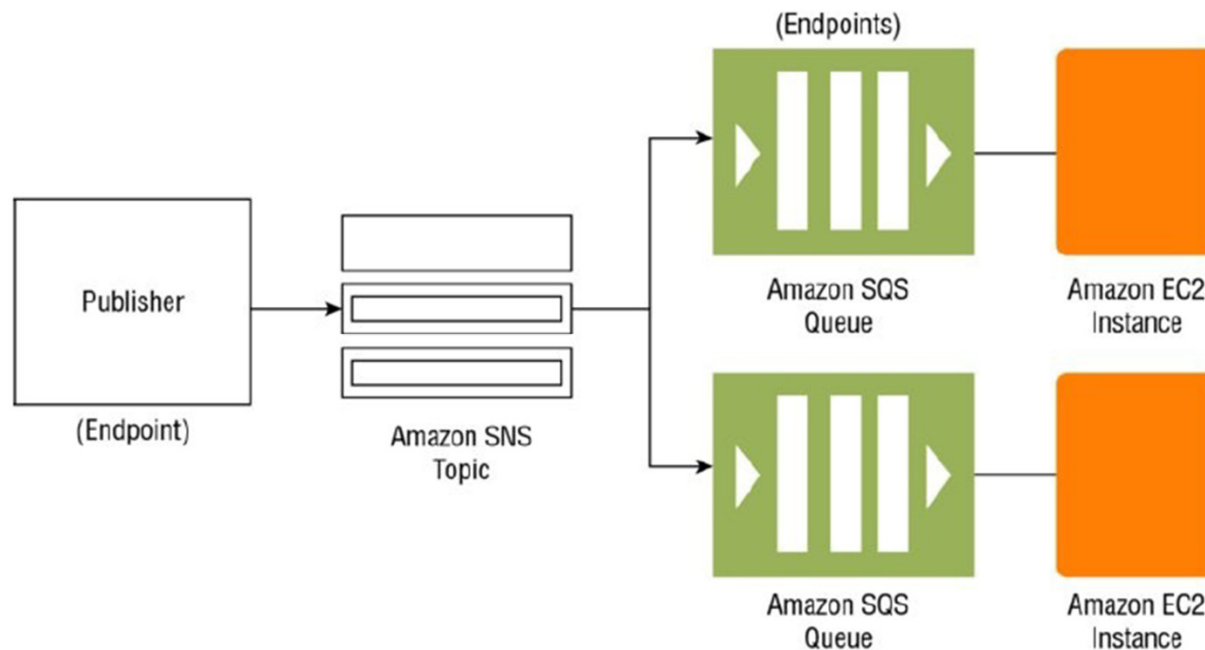
# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- Common Amazon SNS Scenarios
  - Amazon SNS can support a wide variety of needs, including monitoring applications, workflow systems, time-sensitive information updates, mobile applications, and any other application that generates or consumes notifications.
    - Fanout
    - Application and System Alerts
    - Push Email and Text messaging
    - Mobile Push Notifications

# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- Fanout

- A fanout scenario is when an Amazon SNS message is sent to a topic and then replicated and pushed to multiple Amazon SQS queues, HTTP endpoints, or email addresses. This allows for parallel asynchronous processing.



# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- Fanout

- Another way to use fanout is to replicate data sent to your production environment and integrate it with your development environment.
- Expanding upon the previous example, you can subscribe yet another queue to the same topic for new incoming orders.

- Application and System Alerts

- Application and system alerts are SMS and/or email notifications that are triggered by predefined thresholds.

# AMAZON SIMPLE NOTIFICATION SERVICE (SNS)

- Push Email and Text Messaging
  - Push email and text messaging are two ways to transmit messages to individuals or groups via email and/or SMS.
- Mobile Push Notifications
  - Mobile push notifications enable you to send messages directly to mobile applications.
  - The notification message can include a link to download and install the update.



# **AMAZON SIMPLE WORKFLOW SERVICE (SWF)**

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- Amazon SWF makes it easy to build applications that coordinate work across distributed components.
- In Amazon SWF, a task represents a logical unit of work that is performed by a component of your application.
- Coordinating tasks across the application involves managing inter-task dependencies, scheduling, and concurrency in accordance with the logical flow of the application.
- Amazon SWF gives you full control over implementing and coordinating tasks without worrying about underlying complexities such as tracking their progress and maintaining their state.
- Amazon SWF, you implement workers to perform tasks. These workers can run either on cloud infrastructure, such as Amazon EC2, or on your own premises.

## AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- You can create long-running tasks that might fail, time out, or require restarts, or tasks that can complete with varying throughput and latency.
- Amazon SWF stores tasks, assigns them to workers when they are ready, monitors their progress, and maintains their state, including details on their completion.
- To coordinate tasks, you write a program that gets the latest state of each task from Amazon SWF and uses it to initiate subsequent tasks.
- Amazon SWF maintains an application's execution state durably so that the application is resilient to failures in individual components.



# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- **Workflows**

- Using Amazon SWF, you can implement distributed, asynchronous applications as workflows.
- Workflows coordinate and manage the execution of activities that can be run asynchronously across multiple computing devices and that can feature both sequential and parallel processing.
- When designing a workflow, analyze your application to identify its component tasks, which are represented in Amazon SWF as activities.
- The workflow's coordination logic determines the order in which activities are executed.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- **Workflows**
  - **Workflow Domains** - provide a way of scoping Amazon SWF resources within your AWS account - must specify a domain for all the components of a workflow, such as the workflow type and activity types - possible to have more than one workflow in a domain - workflows in different domains cannot interact with one another.
  - **Workflow History** - is a detailed, complete, and consistent record of every event that occurred since the workflow execution started.
  - An event represents a discrete change in your workflow execution's state, such as scheduled and completed activities, task timeouts, and signals.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- **Actors**
  - Amazon SWF consists of a number of different types of programmatic features known as actors.
  - Actors can be workflow starters, deciders, or activity workers - communicate with Amazon SWF through its API.
  - You can develop actors in any programming language.
  - A workflow starter is any application that can initiate workflow executions.
  - Activities within a workflow can run sequentially, in parallel, synchronously, or asynchronously. The logic that coordinates the tasks in a workflow is called the decider.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- **Actors**
  - An activity worker is a single computer process (or thread) that performs the activity tasks in your workflow.
  - Different types of activity workers process tasks of different activity types, and multiple activity workers can process the same type of task.
  - When an activity worker is ready to process a new activity task, it polls Amazon SWF for tasks that are appropriate for that activity worker.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- **Tasks**

- Amazon SWF provides activity workers and deciders with work assignments, given as one of three types of tasks: activity tasks, AWS Lambda tasks, and decision tasks.
- An activity task tells an activity worker to perform its function, such as to check inventory or charge a credit card.
  - The activity task contains all the information that the activity worker needs to perform its function.
- An AWS Lambda task is similar to an activity task, but executes an AWS Lambda function instead of a traditional Amazon SWF activity.
- A decision task tells a decider that the state of the workflow execution has changed so that the decider can determine the next activity that needs to be performed.
  - The decision task contains the current workflow history.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- Task Lists

- Task lists provide a way of organizing the various tasks associated with a workflow.
- You could think of task lists as similar to dynamic queues.
- When a task is scheduled in Amazon SWF, you can specify a queue (task list) to put it in.
- Similarly, when you poll Amazon SWF for a task, you determine which queue (task list) to get the task from.
- Task lists provide a flexible mechanism to route tasks to workers as your use case necessitates.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- Long Polling

- Deciders and activity workers communicate with Amazon SWF using long polling.
- The decider or activity worker periodically initiates communication with Amazon SWF, notifying Amazon SWF of its availability to accept a task, and then specifies a task list to get tasks from.
- Long polling works well for high-volume task processing.
- Deciders and activity workers can manage their own capacity.

# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- Object Identifiers

- Amazon SWF objects are uniquely identified by workflow type, activity type, decision and activity tasks, and workflow execution:
  - A registered workflow type is identified by its domain, name, and version. Workflow types are specified in the call to RegisterWorkflowType.
  - A registered activity type is identified by its domain, name, and version. Activity types are specified in the call to RegisterActivityType.
  - Each decision task and activity task is identified by a unique task token. The task token is generated by Amazon SWF and is returned with other information about the task in the response from PollForDecisionTask or PollForActivityTask.



# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- **Object Identifiers**

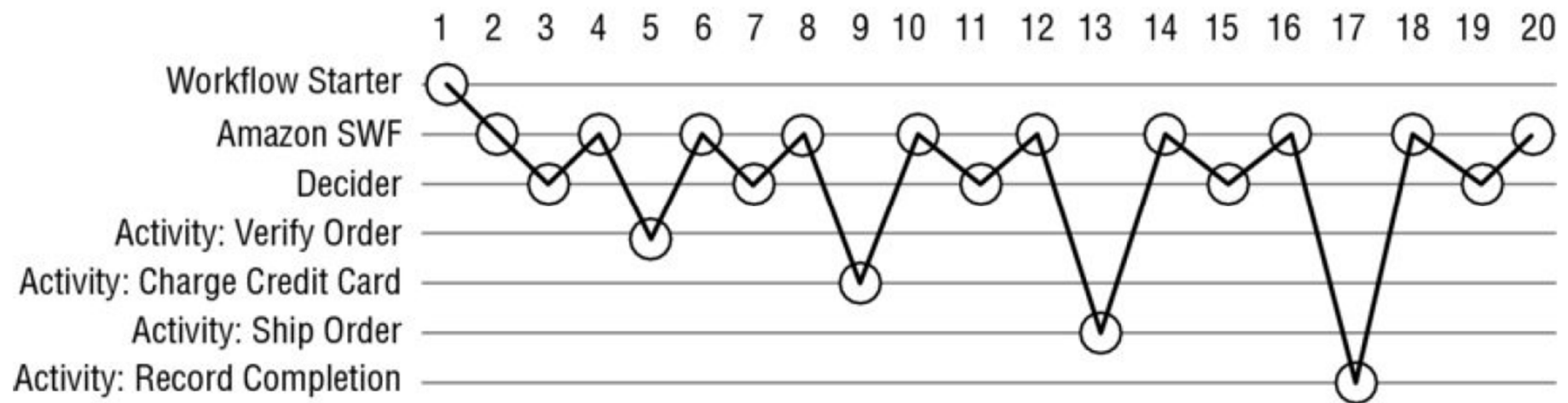
- A single execution of a workflow is identified by the domain, workflow ID, and run ID. The first two are parameters that are passed to StartWorkflowExecution. The run ID is returned by StartWorkflowExecution.

- **Workflow Execution Closure**

- After you start a workflow execution, it is open. An open workflow execution can be closed as completed, canceled, failed, or timed out.
- It can also be continued as a new execution, or it can be terminated.
- The decider, the person administering the workflow, or Amazon SWF can close a workflow execution.

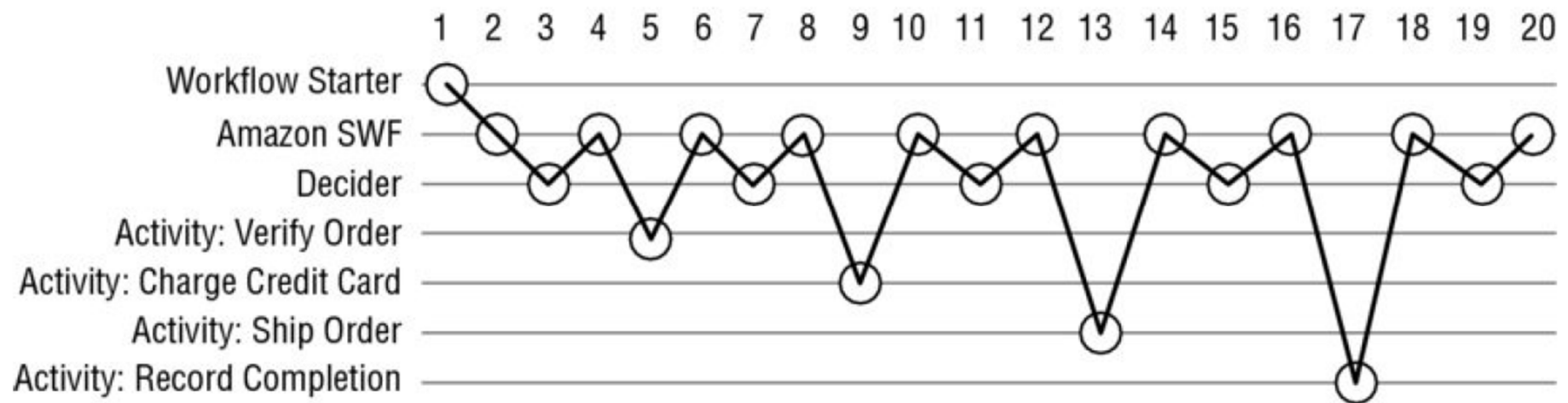
# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- Lifecycle of a Workflow Execution - lifecycle of an order-processing workflow execution from the perspective of components that act on it.



# AMAZON SIMPLE WORKFLOW SERVICE (SWF)

- Lifecycle of a Workflow Execution - lifecycle of an order-processing workflow execution from the perspective of components that act on it.



## **AMAZON SIMPLE WORKFLOW SERVICE (SWF)**

- The following 20 steps describe the workflow detailed in Figure
  1. A workflow starter calls an Amazon SWF action to start the workflow execution for an order, providing order information.
  2. Amazon SWF receives the start workflow execution request and then schedules the first decision task.
  3. The decider receives the task from Amazon SWF, reviews the history, and applies the coordination logic to determine that no previous activities occurred. It then makes a decision to schedule the Verify Order activity with the information the activity worker needs to process the task and returns the decision to Amazon SWF.
  4. Amazon SWF receives the decision, schedules the Verify Order activity task, and waits for the activity task to complete or time out.

## **AMAZON SIMPLE WORKFLOW SERVICE (SWF)**

5. An activity worker that can perform the Verify Order activity receives the task, performs it, and returns the results to Amazon SWF.
6. Amazon SWF receives the results of the Verify Order activity, adds them to the workflow history, and schedules a decision task.
7. The decider receives the task from Amazon SWF, reviews the history, applies the coordination logic, makes a decision to schedule a Charge Credit Card activity task with information the activity worker needs to process the task, and returns the decision to Amazon SWF.
8. Amazon SWF receives the decision, schedules the Charge Credit Card activity task, and waits for it to complete or time out.
9. An activity worker activity receives the Charge Credit Card task, performs it, and returns the results to Amazon SWF.
10. Amazon SWF receives the results of the Charge Credit Card activity task, adds them to the workflow history, and schedules a decision task.

## **AMAZON SIMPLE WORKFLOW SERVICE (SWF)**

11. The decider receives the task from Amazon SWF, reviews the history, applies the coordination logic, makes a decision to schedule a Ship Order activity task with the information the activity worker needs to perform the task, and returns the decision to Amazon SWF.
12. Amazon SWF receives the decision, schedules a Ship Order activity task, and waits for it to complete or time out.
13. An activity worker that can perform the Ship Order activity receives the task, performs it, and returns the results to Amazon SWF.
14. Amazon SWF receives the results of the Ship Order activity task, adds them to the workflow history, and schedules a decision task.
15. The decider receives the task from Amazon SWF, reviews the history, applies the coordination logic, makes a decision to schedule a Record Completion activity task with the information the activity worker needs, performs the task, and returns the decision to Amazon SWF.

## **AMAZON SIMPLE WORKFLOW SERVICE (SWF)**

16. Amazon SWF receives the decision, schedules a Record Completion activity task, and waits for it to complete or time out.
17. An activity worker Record Completion receives the task, performs it, and returns the results to Amazon SWF.
18. Amazon SWF receives the results of the Record Completion activity task, adds them to the workflow history, and schedules a decision task.
19. The decider receives the task from Amazon SWF, reviews the history, applies the coordination logic, makes a decision to close the workflow execution, and returns the decision along with any results to Amazon SWF.
20. Amazon SWF closes the workflow execution and archives the history for future reference.



# **AWS WELL- ARCHITECTED FRAMEWORK**



# AWS WELL-ARCHITECTED FRAMEWORK

- AWS Well-Architected Framework helps you understand the pros and cons of decisions you make while building systems on AWS.
- Using the Framework helps you learn architectural best practices for designing and operating secure, reliable, efficient, cost-effective, and sustainable workloads in the AWS Cloud.
- It provides a way for you to consistently measure your architectures against best practices and identify areas for improvement.
- The process for reviewing an architecture is a constructive conversation about architectural decisions, and is not an audit mechanism.
- We believe that having well-architected systems greatly increases the likelihood of business success.

# AWS WELL-ARCHITECTED FRAMEWORK

- AWS Solutions Architects have years of experience architecting solutions across a wide variety of business verticals and use cases.
- We have helped design and review thousands of customers' architectures on AWS.
- From this experience, we have identified best practices and core strategies for architecting systems in the cloud.
- The AWS Well-Architected Framework documents a set of foundational questions that help you to understand if a specific architecture aligns well with cloud best practices.

# **AWS WELL-ARCHITECTED FRAMEWORK**

- AWS also provides a service for reviewing your workloads at no charge.
- The AWS Well-Architected Tool (AWS WA Tool) is a service in the cloud that provides a consistent process for you to review and measure your architecture using the AWS Well-Architected Framework.
- The AWS WA Tool provides recommendations for making your workloads more reliable, secure, efficient, and cost-effective.



# **DESIGN PRINCIPLE FOR AWS CLOUD ARCHITECTURE**

# AWS DESIGN PRINCIPLES

- The following are design principles for operational excellence in the cloud:
- **Organize teams around business outcomes:**
  - The ability of a team to achieve business outcomes comes from leadership vision, effective operations, and a business-aligned operating model.
  - Leadership should be fully invested and committed to a CloudOps transformation with a suitable cloud operating model that incentivizes teams to operate in the most efficient way and meet business outcomes.
  - The right operating model uses people, process, and technology capabilities to scale, optimize for productivity, and differentiate through agility, responsiveness, and adaptation.
  - The organization's long-term vision is translated into goals that are communicated across the enterprise to stakeholders and consumers of your cloud services. Goals and operational KPIs are aligned at all levels.
  - This practice sustains the long-term value derived from implementing the following design principles.

# AWS DESIGN PRINCIPLES

- **Implement observability for actionable insights:**
  - Gain a comprehensive understanding of workload behavior, performance, reliability, cost, and health.
  - Establish key performance indicators (KPIs) and leverage observability telemetry to make informed decisions and take prompt action when business outcomes are at risk.
  - Proactively improve performance, reliability, and cost based on actionable observability data.

# AWS DESIGN PRINCIPLES

- **Safely automate where possible:**
  - In the cloud, you can apply the same engineering discipline that you use for application code to your entire environment.
  - Can define your entire workload and its operations (applications, infrastructure, configuration, and procedures) as code, and update it.
  - Can then automate your workload's operations by initiating them in response to events. In the cloud, you can employ automation safety by configuring guardrails, including rate control, error thresholds, and approvals.
  - Through effective automation, you can achieve consistent responses to events, limit human error, and reduce operator toil.

# AWS DESIGN PRINCIPLES

- **Make frequent, small, reversible changes:**
  - Design workloads that are scalable and loosely coupled to permit components to be updated regularly.
  - Automated deployment techniques together with smaller, incremental changes reduces the blast radius and allows for faster reversal when failures occur.
  - This increases confidence to deliver beneficial changes to your workload while maintaining quality and adapting quickly to changes in market conditions.
- **Refine operations procedures frequently:**
  - Evolve your workloads, evolve your operations appropriately.
  - Use operations procedures, look for opportunities to improve them. Hold regular reviews and validate that all procedures are effective and that teams are familiar with them.
  - Where gaps are identified, update procedures accordingly. Communicate procedural updates to all stakeholders and teams. Gamify your operations to share best practices and educate teams.



# AWS DESIGN PRINCIPLES

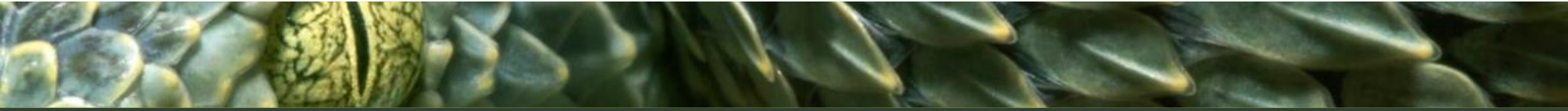
- **Anticipate failure:**
  - Maximize operational success by driving failure scenarios to understand the workload's risk profile and its impact on your business outcomes.
  - Test the effectiveness of your procedures and your team's response against these simulated failures. Make informed decisions to manage open risks that are identified by your testing.
- **Learn from all operational events and metrics:**
  - Drive improvement through lessons learned from all operational events and failures.
  - Share what is learned across teams and through the entire organization.
  - Learnings should highlight data and anecdotes on how operations contribute to business outcomes.
- **Use managed services:**
  - Reduce operational burden by using AWS managed services where possible. Build operational procedures around interactions with those services.

# AWS SERVICES AND FEATURES

- AWS offers a variety of services and features to help with scalability, high availability, fault tolerance, security, and cost optimization:
- **Scalability and elasticity**
  - AWS cloud infrastructure allows businesses to scale up or down to meet demand. Elasticity allows systems to automatically adjust resources based on demand.
- **High availability**
  - To achieve high availability, you can:
    - Design systems to avoid single points of failure (SPOF)
    - Use automated monitoring, failure detection, and failover mechanisms
    - Create an Auto Scaling Group that spans multiple Availability Zones
    - Add an Elastic Load Balancer in front of an Auto Scaling Group

# AWS SERVICES AND FEATURES

- **Fault tolerance**
  - Fault tolerance means that a system will maintain uptime even during a primary system outage. To improve fault tolerance, you can configure your load balancer in multiple Availability Zones in the same Region.
- **Security**
  - AWS offers robust security measures.
- **Cost optimization**
  - AWS offers a variety of pricing options and management tools to help you optimize costs. You can:
    - Use reserved instances for committed usage
    - Use spot instances for flexible workloads
    - Select cost-effective storage options
    - Use continuous monitoring to identify cost-saving opportunities
    - Set up alerts to quickly identify and address inefficiencies



## future initiatives

1. **Product enhancement.** Introduce regular updates and features to enhance product offerings.
2. **Technology integration.** Explore emerging technologies for potential integration into our operations.
3. **Collaborative partnerships.** Foster collaborations with tech innovators and industry leaders to drive innovation.

Green supply chain  
Reduced carbon footprint  
Waste reduction  
Water conservation

# Module 7

BCSE355L – AWS Solution Architect

